ChatGPT said:

Variometer failure-mode research: What we should avoid building

Your suspicion is supported by published developer documentation and pilot reports:

A variometer can be quick to detect lift, yet slow to recognize that the lift has ended. Increasing filtering to suppress noise can make both problems worse.

The more surprising discovery is that the pressure sensor and Kalman filter are not the only sources of delay. **The audio-beep scheduler can introduce its own delay even when the velocity estimate is already correct.**

BlueFlyVario's developer explicitly documents this behaviour, and XCTrack users have reported cases where the incoming sensor data appears responsive but the sound lags behind.

Blue Fly Var... +1

1. Problems discovered in existing variometers

These are documented behaviours, design limitations and user-reported issues—not all confirmed software defects.

Priority	Failure mode	Evidence	Impact on our product
Critical	Filtering delays lift onset	BlueFly documentation; XCTrack feature discussion	Late thermal detection
Critical	Beeping continues after lift ends	BlueFly beep-scheduling design	Misleading audio while exiting lift
Critical	App introduces additional audio delay	XCTrack pilot report and issue tracker	USB improvements could be wasted
High	Weak signals require smoothing to avoid false beeps	BlueFly filter documentation	Noise versus sensitivity conflict
High	IMU bias and orientation errors	Open-source Kalman implementation	False acceleration and lift readings
High	Thermal/airflow pressure artifacts	Sensor/port engineering concern	False climbs or sink
High	Aircraft pitching creates misleading vertical-speed cues	XC Tracer TEC explanation	Confuses energy exchange with rising air
High	App applies additional filtering to external sensor data	ESP32 open-source vario developer	Double-filtering and added lag
Medium	Audio threshold oscillation	Documented threshold/hysteresis design	Chattering at weak-lift boundary

Medium	Weak-lift audio has delayed or unclear updates	XCTrack developer issue	Missing subtle changes in lift
Medium	Annoying tone character and abrupt beeps	Pilot reports on XC Tracer and other models	Fatigue and reduced usefulness
Medium	Two instruments indicate conflicting lift/sink	Pilot reports and differing estimator response	Difficult to interpret during flight

Blue Fly Var... +5

The first four are the ones I would treat as product-defining engineering risks.

2. The persistent-beeping problem is real

This is the most directly relevant discovery.

BlueFlyVario's developer describes an audio state machine where crossing the lift threshold schedules a beep for a calculated duration, followed by a silent interval. The decision to end beeping occurs only after the scheduled sequence has progressed.

Consequently, the audio can continue after the estimated climb rate has already dropped below the lift-off threshold.

Blue Fly Vario ...

Illustration: actual lift versus an audio scheduler that finishes its current beep

There are actually two independent delays:

1. **Estimator release delay:** The filter continues reporting positive vertical velocity after the actual climb has stopped.
2. **Audio release delay:** The audio scheduler continues playing a previously scheduled beep after the estimated climb has stopped.

These add together.

A faster barometer does not fix the second problem.

How we should design ours

I would implement an interruptible audio scheduler.

When the estimator determines that lift has ended, it should be able to terminate the current tone immediately or with a very short amplitude ramp to prevent audible clicks.

We should not wait for an entire beep-and-silence cycle to finish.

But we also need to avoid the opposite problem: sound cutting in and out rapidly whenever the climb rate fluctuates around zero.

That suggests **asymmetric audio behaviour**:

- Fast onset when credible lift is detected.
- Fast release when lift genuinely ends.
- Carefully tuned hysteresis or confidence-based gating to prevent chatter.
- Short attack/release ramps for acoustic smoothness, without holding an obsolete lift indication.

This is a stronger design direction than simply making one Kalman filter more aggressive.

3. An important finding: developers have encountered this exact noise–latency trade-off

In XCTrack issue #652 ↗, a developer discussion specifically addresses whether accelerometer data could reduce the latency introduced by barometric filtering.

Interestingly, XCTrack's developer explained that experiments with phone IMUs had been unsuccessful, including a later attempt on more modern phones. Another contributor pointed toward an open-source accelerometer-assisted variometer.

xcontest-public…

This reinforces two points:

- IMU fusion is promising, but it is not automatically better.

- Sensor calibration, mounting, gravity compensation and estimator tuning determine whether the IMU actually reduces delay.

The open-source Kalmanfilter_altimeter_vario ↗ project is particularly useful. It estimates altitude, vertical velocity and residual accelerometer bias. Its documentation explicitly acknowledges that bias changes with environmental conditions.

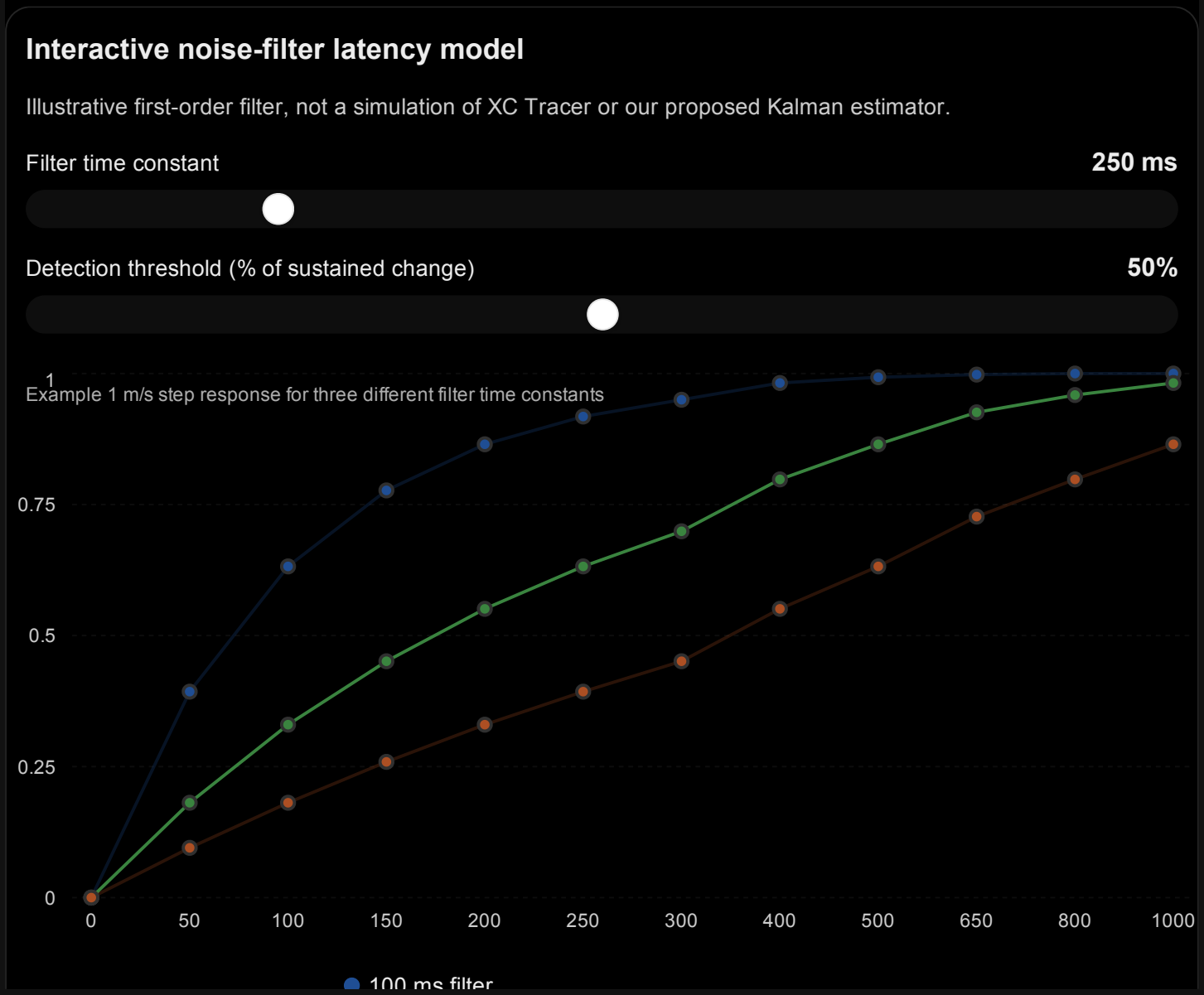
GitHub

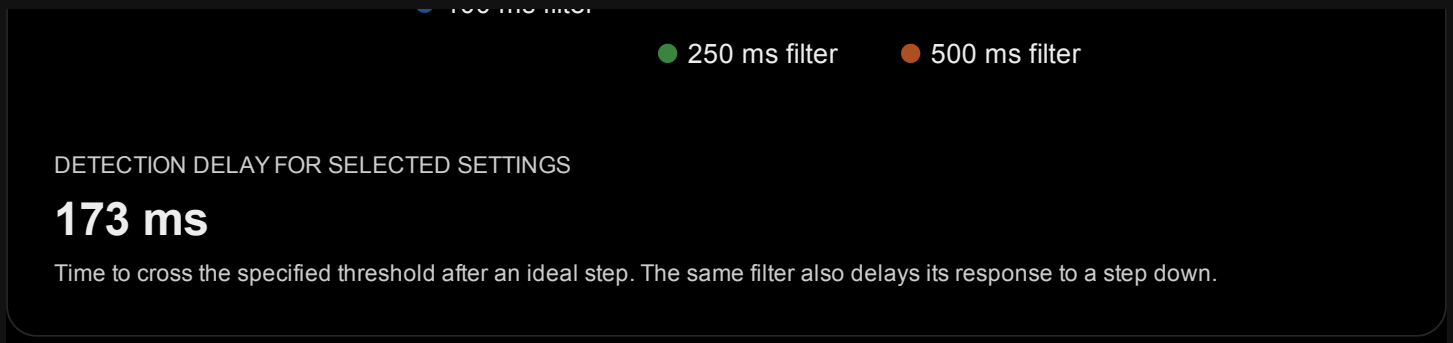
A separate ESP32 IMU/barometer variometer ↗ provides real recorded data and a comparison of pressure-only versus IMU-fused estimators. The author also preferred the MS5611's audio smoothness to the BMP388 in their implementation—an experience report, not a universal sensor ranking.

GitHub

4. Filtering: how much latency does smoothing create?

Consider a basic first-order low-pass filter. Its response to a sudden sustained change illustrates the trade-off.





This demonstrates why simply smoothing harder is a poor solution to noisy raw pressure. A filter with a long memory can continue showing positive climb well after the physical climb has ended.

Kalman filters can perform better by incorporating inertial measurements, but their actual performance depends on calibration and tuning. A Kalman filter is not inherently lag-free.

5. Additional failure modes our design should explicitly test

The following are engineering risks, not necessarily confirmed faults in a particular commercial instrument.

Failure	Proposed validation
Pressure inlet affected by wind or suction	Change orientation and impose controlled airflow at constant altitude
Heat from the phone creates slow pressure drift	Compare USB-powered operation with phone heating and cooling
Mechanical pressure spikes when touching device	Tap and move housing while recording raw data
IMU tilt interpreted as vertical acceleration	Rotate without changing altitude
Gyro/accelerometer bias accumulates	Log long stationary sessions at different temperatures
Large sink-to-lift transitions create overshoot	Replay synthetic and measured pressure changes
Old sensor samples delivered late over USB	Introduce USB congestion and record sample age
Audio remains active after stale measurements	Disconnect sensor input during a synthetic climb
Multiple sensors disagree	Disconnect one and inject controlled offsets
Algorithm appears sensitive but has excessive false alarms	Compare against independent reference data over stationary and flight-like tests

There is another noteworthy problem: **vertical motion does not necessarily mean rising air**. XC Tracer documents how speed-bar release and pitch changes can create apparent climb from kinetic-to-potential-energy conversion, and uses total energy compensation in some models to address it. Our IMU cannot completely solve this without additional information about airspeed and aircraft dynamics.

XC Tracer

6. Proposed performance requirements for our prototype

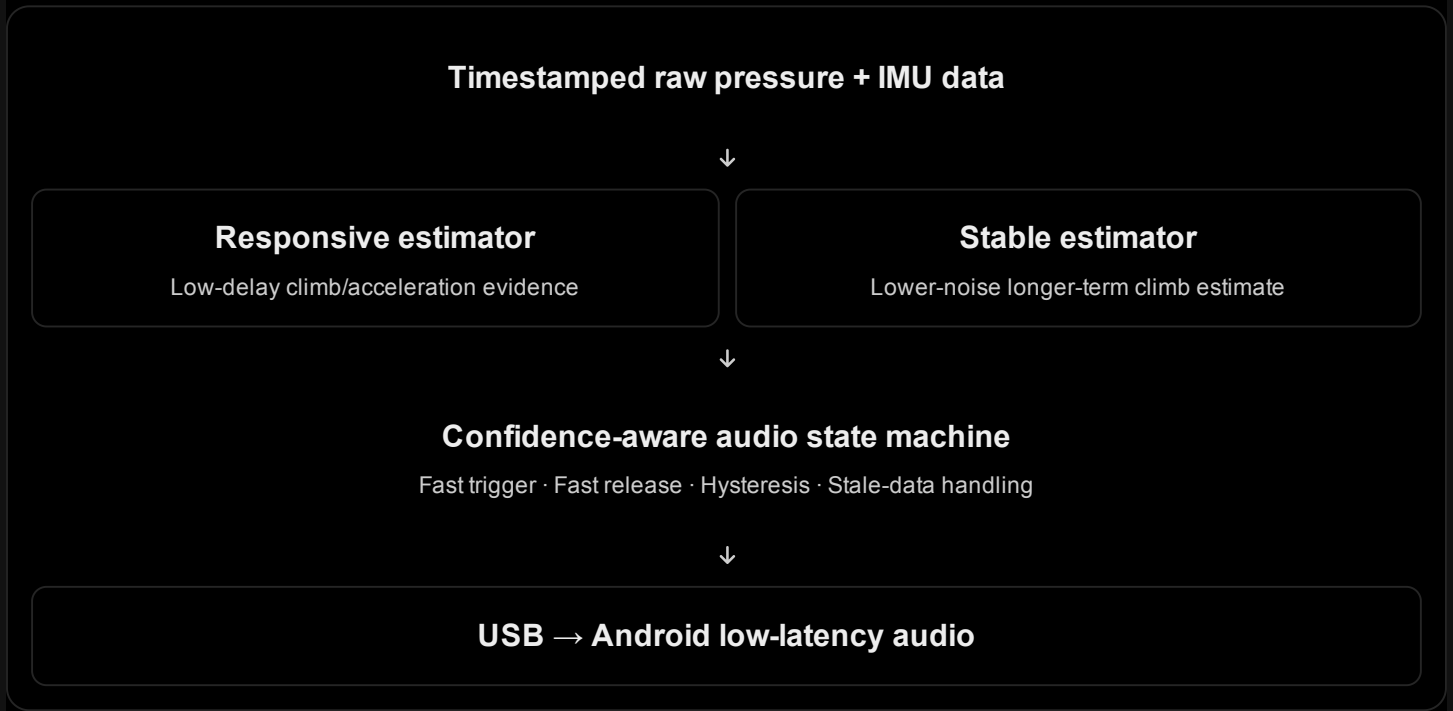
These are suggested engineering targets, not demonstrated capabilities.

Metric	Initial target
Pressure acquisition	50–100 Hz, configuration-dependent
IMU acquisition	200 Hz
Estimator update	At least 100 Hz
Onset latency	Under 150 ms for defined test input
Release latency	Under 150 ms for defined test input
Additional audio-release latency	Under 30 ms after estimator decision
USB sample loss	0 during nominal sustained operation
False-beep rate	Measure baseline first; set acceptance limit from evidence

We should measure onset and release separately, at several signal strengths.

For example, a +0.2 m/s simulated climb may produce a very different onset delay from +2.0 m/s. Reporting one universal response time would conceal that difference.

7. My preferred architecture after reviewing these findings



This dual-estimator architecture is a hypothesis worth testing against a simpler well-tuned single estimator, not something I would commit to blindly.

I would also ensure the app uses the timestamped velocity estimate directly for audio decisions instead of adding another large smoothing filter.

Research sources most worth preserving

- BlueFlyVario — Hardware Settings ↗: actual filtering and audio threshold/scheduler behaviour.
- XCTrack issue #225 ↗: reports of delayed audio and the developer's tuning changes.
- Skydrop + XCTrack delay discussion ↗: raw sensor responsiveness versus sound responsiveness.
- Kalmanfilter_altimeter_vario ↗: bias-aware barometer/IMU estimation.
- ESP32 variometer implementation ↗: recorded data, source code and offline filter comparisons.
- XC Tracer TEC explanation ↗: misleading climb indications caused by aircraft energy exchange.

My principal recommendation: Treat *exit-from-lift response* as a first-class requirement, with its own tests and acceptance criteria. A responsive instant vario should not only tell us when lift arrives—it should also promptly stop telling us lift exists when it no longer does.

For now, your ESP32-S3, MS5611 and MPU6050 provide the right experimental hardware to investigate all of these issues before selecting the final sensors or PCB architecture.